El flujo diario seguro (memorízate este)

```
# 1. Empieza el día actualizado
git checkout mi-rama
git pull
# 2. Trabaja normal... editas archivos en VS Code...
# 3. Revisa qué cambió
git status
# 4. Guarda tu trabajo
git add .
git commit -m "feat: lo que hiciste"
# 5. Sube al remoto
git push
```

Eso es el 90% de tu vida con Git. El resto son casos especiales.

Cuando quieres mergear tu rama a otra (ej. `feature` → `main`)

```
# 1. Asegúrate de tener tu rama subida y al día
git checkout mi-rama
git pull
git push
# 2. Ve a la rama destino y actualízala
git checkout main
git pull
# 3. Trae tu rama
git merge mi-rama
# 4a. Si NO hay conflictos:
git push
# 4b. Si HAY conflictos:
#     - Resuelve en VS Code (panel Source Control, Ctrl+Shift+G)
git add .
git commit          # cierra el merge
git push
```

Regla mnemotécnica: *"Voy a la casa que recibe la visita."* Para meter A en B → te pones en B.

Mnemotecnia base: **RAC-P**

Letra	Comando	Cuándo
R	`git status` (**R**evisar)	Constantemente, es gratis
A	`git add .`	Cuando terminas algo
C	`git commit -m "..."`	Después de add
P	`git push`	Cuando quieres compartir

Y al empezar el día: `git pull` para arrancar al día.

Frase: *"Pull, trabajo, Revisa, Add, Commit, Push."*

Las 3 reglas que te ahorran el 80% de los líos

1. **`status` entre cada paso.** Es gratis y te dice exactamente dónde estás.
2. **Antes de mergear, las dos ramas al día con `pull`.**
3. **Para meter A en B → te pones en B.** Nunca al revés.

Sintaxis: `origin mainV2` vs `origin/mainV2`

- **Con espacio** (`origin mainV2`) → solo en `fetch` / `push` / `pull`. Son dos argumentos: el remoto Y la rama.
- **Con barra** (`origin/mainV2`) → en `merge` / `rebase` / `checkout` / `log`. Es una sola cosa: la rama remota.

Truco: *"Si hablo **CON** el remoto, espacio. Si hablo **DE** una rama remota, barra."*

Anexo: casos especiales (poco frecuentes)

Primera vez que subes una rama nueva

```
git push -u origin nombre-rama
```

El `-u` solo hace falta una vez. Después ya es `git push` a secas.

Crear una rama nueva

```
git checkout -b nombre-rama
```

Cambiarte de rama

```
git checkout nombre-rama
```

Ver todas tus ramas

```
git branch
```

Aplastar una rama con otra (sobrescribir, perder commits)

Solo si sabes lo que haces y nadie más trabaja en esa rama:

```
git checkout rama-destino
git reset --hard rama-origen
git push --force-with-lease
```

Nunca uses `--force` a secas. `--force-with-lease` te protege si alguien subió cambios mientras tanto.

Abortar algo que salió mal

```
git merge --abort      # si estás en mitad de un merge
git rebase --abort     # si estás en mitad de un rebase
```

Te devuelve al estado anterior, sin daño.

Ver el historial reciente

```
git log --oneline -10
```

Traer cambios del remoto sin aplicarlos

```
git fetch origin
```

Útil para ver qué hay nuevo antes de hacer `pull` o `merge`.

❌ Errores típicos a evitar

- ❌ `git fetch origin/rama` → mal, `origin/rama` no es un remoto. Es `git fetch origin rama`.
- ❌ `git merge origin rama` → mal, dos argumentos. Es `git merge origin/rama`.
- ❌ Estar en rama A y hacer `merge B` esperando que B reciba los cambios. **Es al revés.**
- ❌ Hacer `--force` sin `--with-lease`. Puede borrar trabajo de compañeros.
- ❌ `git rebase origin rama` → te cambia de rama sin avisar y rebasea la equivocada.

🆘 Resolver conflictos en VS Code (paso a paso)

1. Abre el panel **Source Control** con `Ctrl+Shift+G`.
2. En la sección **Merge Changes** verás los archivos en conflicto.
3. Haz clic en cada archivo. Verás bloques así:

   ```
   <<<<<<< HEAD
   (lo que hay en tu rama actual)
   =======
   (lo que viene de la otra rama)
   >>>>>>> otra-rama
   ```

4. Encima de cada bloque tienes botones:
   - **Accept Current Change** → te quedas con lo de tu rama actual.
   - **Accept Incoming Change** → te quedas con lo de la otra rama.
   - **Accept Both Changes** → mezcla ambas (luego limpia a mano).
   - **Compare Changes** → ver diferencias lado a lado.
5. Guarda el archivo (`Ctrl+S`).
6. Marca como resuelto: `git add nombre-archivo` (o `git add .` para todos).
7. Cuando estén todos resueltos: `git commit` para cerrar el merge.

📖 Glosario rápido

Término	Qué es
Rama (branch)	Una línea de trabajo paralela.
Commit	Un punto guardado en la historia.
Remote / origin	El repositorio de GitHub (o donde sea).
Push	Subir tus commits al remoto.
Pull	Bajar y aplicar commits del remoto.
Fetch	Bajar info del remoto sin aplicarla.
Merge	Juntar dos ramas en una.
HEAD	La rama en la que estás ahora.
Upstream	La rama remota a la que está vinculada tu rama local.